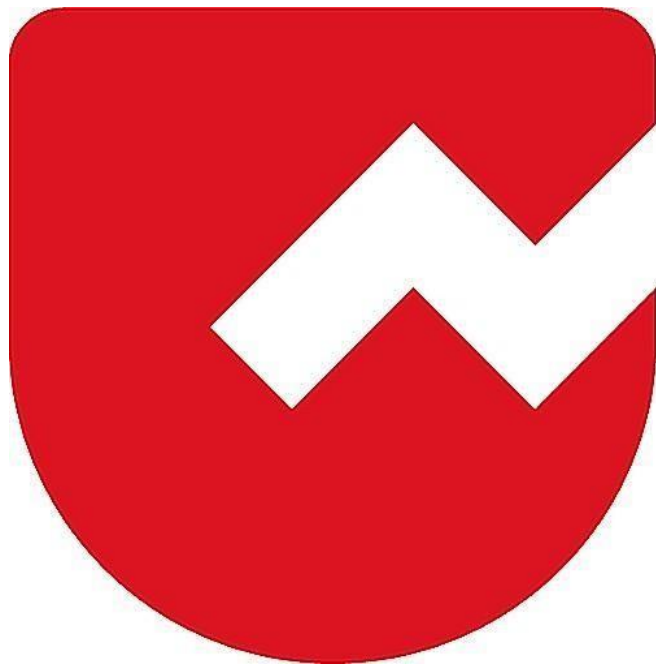




Zoho Schools for Graduate Studies



Notes

Day 9 -Unconditional Control Statements

Unconditional Control Statements:

Those statements that change or alter the flow of control unconditionally are referred as unconditional control statements.

Examples: 1.break, 2.continue and 3.return.

1.what a 'continue' statement does?

It abruptly terminates the current iteration of the loop and proceeds with the next iteration.

- It cannot be used inside a “if” statement without loop.
- After “continue” no statements are allowed.

Example:

```
for(int i=1;i<=10;i++) {  
    if(i==4 | i==8) continue;  
    System.out.println(i);  
}
```

output: 1

2

3

5

6

7

9

10

what a labeled 'continue' statement does?

It abruptly terminates the current iteration of the labeled loop and proceeds with the next iteration of the labeled loop.

Example:

```
outer: for(int i=1; i<=3; i++){  
    Inner: for(int j=1; j<=3; j++){
```

```
        if(i==j) continue outer;
```

```
        System.out.println(i+ “ ” +j);
```

```
    }}
```

output:

2 1

3 1

3 2

2.what a 'break' statement does?

It abruptly terminates the current loop and takes the flow of control outside the current loop.

Example:

```
for(int i=1;i<=10;i++) {  
    if(i==4) break;  
    System.out.println(i);  
}
```

output: 1

2

3

what a labeled 'break' statement does?

It abruptly terminates the labeled loop and takes the flow of control outside the labeled loop.

Example:

```
outer: for(int i=1; i<=3; i++){  
    Inner: for(int j=1; j<=3; j++){  
        if(i==j) continue Inner;  
        System.out.println(i+ " " +j);  
    }  
}
```

output:

2 1

3 1

3 2

3.what a 'return' statement does?

It takes the flow of control from the current method to the called method.

Example:

```
int n=3;

System.out.println(go(5) + n);

}

static int go(int x) {

int y=4;

return x+y;

}
```

output: 12

what is the output of the following code :

```
int nTerms=5;

int sum=0;

for(int i=0;i<nTerms;i++){

if(i%2==1){

nTerms++;

continue;

} else {

sum+=i;

}

System.out.print(sum);

} // output : 20
```

Reason:

This final example shows a for loop in which the loop control variable (nTerms) is modified inside the loop body. *Each time the condition $i \% 2 == 1$ is true, the value of nTerms is increased.* This extends the range of the loop and causes more iterations to be executed than originally expected. During the even iterations, the value of i is added to sum and printed. As a result of these extended iterations, the loop finally ends with the accumulated value 20 being printed last.(here continues is meaningless).what is

the output of the following code :

```
int i=10;

outer: while(true){
    if(i==0){
        i--;
        break outer;
    }

    System.out.print(i+" ");
} }
```

Output:infinite Loops

Reason:

This final example shows a while(true) loop with a labeled break (break outer). The condition if(i == 0) is never satisfied because the value of i is not being changed inside the loop body when it is greater than 0. As a result, the statement System.out.print(i + " "); keeps executing with the same value 10. Since i remains unchanged, the loop never terminates and the value 10 is printed infinitely.

what is the output of the following code :

```
int i=10;

outer: while(true){
    if(i==0){
        continue outer;
    }

    i--;
}

System.out.print(i+" ");
} } //output: compiler error
```

Reason:

Any code written after the continue inside the same block becomes unreachable.

what is the output of the following code :

```
int sum=0;

for(int i=0;i<5;i++){
    for(int j=0;j<5;j++){
        if(i==j)
            break;
        else sum+=i+j;
    } } System.out.println(sum);
}
```

output: 40

Reason:

This final example shows a nested `for` loop with a `break` statement. The inner loop terminates whenever `i == j`. For each value of `i`, the loop only adds pairs `(i + j)` until `j` becomes equal to `i`. After that, the `break` immediately ends the inner loop, and control returns to the outer loop.

Because of this early termination, not all combinations of `i` and `j` are added—only those where `j < i`. The accumulated result of these additions produces a final `sum` value of 40, which is printed.